NAZARBAYEV UNIVERSITY

Programming for Engineers

Week 9: Classes, Information Hiding, and Inheritance

Yanwei Wang
School of Engineering
Nazarbayev University

Fall 2026



Week 9 at a glance

Lecture 25

Getters, setters, and information hiding

- Public interfaces.
- Validated updates.
- Protected invariants.

Lecture 26

Class variables and shared state

- Ordinary lookup.
- Instance overrides.
- Mutable-state traps.

Lecture 27

Inheritance and specialization

- Parent and child.
- Initialization and overrides.
- Compatible reuse.

Define the interface $\rightarrow$ Protect the state
$\rightarrow$ Share deliberately $\rightarrow$ Specialize compatibly

Course scope: Python classes and inheritance

Official Session 9 content

More on classes; getters and setters; information hiding; class variables; inheritance.

This week's foundation

Week 8 objects and methods, Week 7 validation and testing, and Week 5 references and mutation.

Optional, not required for core examples

Properties, name-mangling demonstration, and string representations. No multiple inheritance or abstract-class machinery.

ENG 101 Course Specification Report, p. 4, Session 9; CLOs 1 and 5.

NAZARBAYEV UNIVERSITY

LECTURE 25

Getters, Setters, and Information Hiding

Control access to state without hiding its engineering meaning

NAZARBAYEV UNIVERSITY

Lecture 25 learning goals

1. Explain the difference between a public interface and stored representation.
2. Read and change state through explicit getters and setters.
3. Validate a proposed update before committing it.
4. Interpret single- and double-leading underscores accurately.
5. Recognize when a returned list exposes mutable internal data.
6. Test both returned values and the state after failure.

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Warm-up: a check can be bypassed

```
from access_models import PublicPressureReading
p = PublicPressureReading(315.0)
# Deliberate misuse: direct assignment bypasses
    the checked update method.
p.value_kpa = -5.0
print(p.value_kpa)  # The program has stored a
    contract-violating value.
```

Predict before running

1. Why does this assignment run?
2. What happened to the promised nonnegative pressure?
3. Did the checked update method run?

Individual then pairs - 3 min

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Write an invariant before choosing an interface

Object	Condition that should remain true
Absolute-pressure reading	Finite pressure $p \geq 0$, reported in kPa
Ideal resistor	Finite resistance $R > 0$, in ohms
Sensor interval	Finite limits with lower $\leq$ upper
Flat reading history	Every stored element is a finite numeric reading

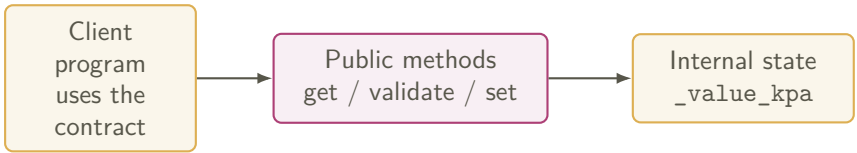
Invariant

A condition promised for each valid stored state, before and after supported operations.

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Hide the representation, not the meaning



Information hiding

Client code depends on documented operations and units, not on the exact internal attribute names.

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Public and non-public names communicate intent

Name form	Meaning in these examples
`get_value_kpa()`	Public operation intended for client code
`_value_kpa`	Non-public implementation detail by convention
`__value`	Name mangling helps avoid accidental subclass collisions
`__init__`	Special method name defined by Python

Do not overclaim

Neither one underscore nor name mangling encrypts data or provides a security boundary.

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

A getter returns the promised quantity

```
    def get_value_kpa(self):
        """Return the current pressure in kPa
    without changing the object."""
        return self._value_kpa
```

Public contract

- Return one pressure value in kPa.
- Do not change the stored reading.
- Let the caller decide whether to print it.

Client expression

```
pressure = p.get_value_kpa()
```

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

A setter checks before it changes state

```
    def set_value_kpa(self, proposed_kpa):
        """Accept a valid absolute pressure; on failure preserve the old value."""
        # Validate the proposal before assigning any stored attribute.
        new_value = checked_nonnegative(proposed_kpa, "Absolute pressure")
        self._value_kpa = new_value
        # No explicit return: a successful update returns None.
```

State guarantee

When validation raises an exception, the assignment has not occurred. The previous valid reading remains stored.

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Initialize through the same validated rule

```
class PressureReading:
    """Store one finite, nonnegative absolute
    pressure; public units are kPa."""
    def __init__(self, value_kpa):
        # This standalone teaching class
    initializes through its own setter.
        self.set_value_kpa(value_kpa)
```

One rule, two uses

- Construction must establish a valid initial reading.
- Later updates must maintain the same rule.
- This standalone class uses its setter in initialization.

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

A familiar helper supplies validation

Proposal	Result of the helper
315 or 315.0	Return 315.0
0	Return 0.0
-5	ValueError: negative absolute pressure
`"315"` or `True`	TypeError under the chosen numeric contract
NaN or infinity	ValueError: not finite

No hidden dependency

`checked_nonnegative(value, label)` is included in `examples/numeric_checks.py`; complete helpers are in that file, with finite checks in the reference slides.

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Use the interface; inspect the returned result

```
from access_models import PressureReading
p = PressureReading(315.0)
print(p.get_value_kpa())  # Read through the
    documented interface.
result = p.set_value_kpa(400.0)  # Change the
    same object's state.
print(p.get_value_kpa())
print(result)  # A successful setter returns None
    , not a replacement object.
```

Observed output

```
315.0
400.0
None
```

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

Trace a rejected state change

```
from access_models import PressureReading
p = PressureReading(315.0)
try:
    p.set_value_kpa(-5.0)  # Invalid absolute
    pressure under this contract.
except ValueError:
    print("Rejected")
print(p.get_value_kpa())  # The old reading must
    still be present.
```

Two separate observations

```
Rejected
315.0
```

Reason

The rejected proposal is not a replacement reading.

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Tests must cover both acceptance and rejection

Case	Expected behavior	Stored value afterward
Initialize at 0	Accepted	0.0 kPa
Set 400	Accepted	400.0 kPa
Set 900	Accepted numeric data	900.0 kPa
Set -1	Rejected	Previous reading
Set text or NaN	Rejected	Previous reading

Model boundary

Nonnegative is not the same as within an operating limit. These tests check the stated data contract.

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Retain a regression check for state preservation

```
from access_models import PressureReading
p = PressureReading(0.0)  # Test the included zero boundary first.
assert p.get_value_kpa() == 0.0
p.set_value_kpa(900.0)  # High pressure is valid data, not automatically safe.
assert p.get_value_kpa() == 900.0
before = p.get_value_kpa()
try:
    p.set_value_kpa(-1.0)
except ValueError:
    assert p.get_value_kpa() == before
else:
    raise AssertionError("Negative absolute pressure was not rejected")
print("Access tests passed")
```

Trace the successful and rejected routes - 3 min

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

Printing inside a getter breaks reuse

```
# Deliberate error: printing a value is not returning a
      value.
class Pressure:
    def __init__(self):
        self._value = 315.0
    def get_value(self):
        print(self._value)  # Repair: return self.
    _value.
p = Pressure()
pressure_pa = p.get_value() * 1000.0  # None cannot be
      used as a numeric reading.
```

Diagnose the consequence

- The getter prints 315.0.
- It then returns None.
- The multiplication raises TypeError.
- Repair the getter by returning the value.

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

A getter can accidentally expose a mutable list

```
# Deliberately weak interface: a getter hands out the
    internal mutable list.
class ReadingSeries:
    def __init__(self):
        self._readings = [20.0, 21.0]
    def get_readings(self):
        return self._readings
series = ReadingSeries()
returned = series.get_readings()
returned.append(999.0)  # This also changes the object'
    s internal data.
print(series.get_readings())
```

What does the client receive?

A reference to the same list, not an independent snapshot.

Predict the final list - 3 min

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Return a copy when a snapshot is promised

```
from access_models import ReadingSeries
series = ReadingSeries([20.0, 21.0])
returned = series.get_readings()  # This call
    returns a new outer list.
returned.append(999.0)
print(returned)
print(series.get_readings())  # The stored
    readings remain unchanged.
```

Observed output

```
[20.0, 21.0, 999.0]
[20.0, 21.0]
```

Scope of this repair

A shallow copy is enough for this flat list of immutable numeric elements.

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

A stable interface can hide different storage

```
    def get_value_kpa(self):
        """Return the current pressure in kPa
    without changing the object."""
        return self._value_kpa
```

Alternative storage

`PressureStoredPa` stores `_value_pa`; its getter returns `_value_pa / 1000.0`.

Unchanged promise

Client methods still accept and return kPa.

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

One client can use either representation

```
from access_models import PressureReading, PressureStoredPa
# The same client statements work with two internal representations.
for model in [PressureReading, PressureStoredPa]:
    p = model(315.0)
    p.set_value_kpa(400.0)
    print(p.get_value_kpa())  # Both interfaces promise kPa, not storage units.
```

Both runs print 400.0

The client never reads either backing attribute. Test the same public behavior after refactoring.

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Write the public contract as a small table

Operation	Promise	State change?
`get_value_kpa()`	Return current pressure in kPa	No
`set_value_kpa(x)`	Accept valid proposal; return None	Yes, on success
Rejected setter call	Raise; retain old reading	No
`get_readings()`	Return a fresh flat numeric list	No

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Pair task: design controlled resistance access

An ideal resistor model

Store one finite resistance $R > 0$ in ohms. Provide a getter and a setter. Reject zero or negative resistance without changing an existing valid value.

Write

1. A class and two method headers.
2. The validation before assignment.
3. One accepted and one rejected test.

Pairs - 5 min

Explain

Why is zero treated differently from the absolute-pressure example?

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Debugging clinic: inconsistent attribute names

```
# Deliberate logic bug: setter and getter use different
      attribute names.
class Reading:
    def __init__(self):
        self._value = 20.0
    def get_value(self):
        return self._value
    def set_value(self, value):
        self.value = value  # BUG: missing underscore.
r = Reading()
r.set_value(30.0)
print(r.get_value())  # Still 20.0: no exception
     exposes this mistake.
```

The program runs

1. Predict the printed value.
2. Which attribute did the setter change?
3. Make the smallest repair.

Individual - 3 min

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

Do not add accessors mechanically

Useful controlled access

- Preserve an invariant.
- Convert a documented representation.
- Return a snapshot instead of shared mutable data.

Unnecessary ceremony

- Many trivial getters for an uncomplicated record.
- A setter for a value that should not be editable.
- A query that unexpectedly changes state.

Another Python option

Properties provide attribute-like access to methods; they are shown in the optional reference slides.

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

An information-hiding checklist

1. Document public operations, units, and side effects.
2. Keep implementation names out of ordinary client code.
3. Validate every supported state-change route.
4. Check what references a getter returns.
5. Test that failure preserves the intended state.
6. Treat underscore conventions as cooperation, not security.

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

Lecture 25 exit ticket

Without running Python

A pressure reading contains 315 kPa. A setter rejects -5 kPa. State the exception type, the value returned by the next getter, and the reason the old reading survives.

Two short explanations

- Does `_value_kpa` prevent all external access?
- Why might `get_readings()` return a copy?

Individual - 3 min

Wang, Appendix H.8, H.12–H.14; Guttag, Section 8.3. Teaching adaptations.

NAZARBAYEV UNIVERSITY

LECTURE 26

Class Variables and Shared State

Decide what belongs to each instance and what is deliberately shared

NAZARBAYEV UNIVERSITY

Lecture 26 learning goals

1. Distinguish class attributes from instance attributes.
2. Trace ordinary attribute lookup and instance shadowing.
3. Change a shared default intentionally.
4. Recognize an accidentally shared mutable list.
5. Implement and interpret a simple creation counter.
6. Test independent state and restore shared test settings.

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

NAZARBAYEV UNIVERSITY

Warm-up: which data should be shared?

Information	One object or the class?
The current tank temperature	One particular sensor
The current pipe pressure	Another particular sensor
Default report decimal places	A shared reporting preference
History of one experiment	A separate list for that experiment
Number of successful creations	A deliberately shared counter

Pairs - 3 min

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

NAZARBAYEV UNIVERSITY

Define an ordinary class attribute

```
class SensorFormat:
    """An ordinary class setting provides the default report precision."""
    decimal_places = 2  # Shared default, not each sensor's measured value.

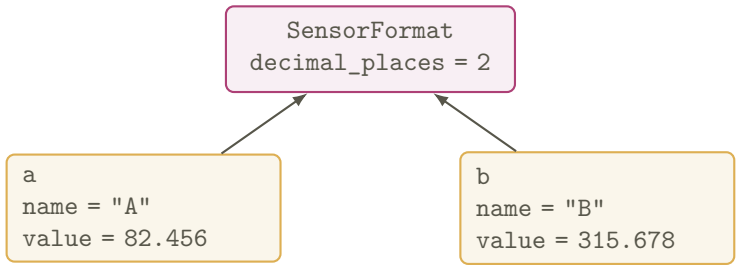
    def __init__(self, name, value):
        self.name = name
        self.value = value  # Teaching assumption: a finite numeric reading.
```

Read the indentation

`decimal_places` is assigned in the class body. `self.value` is assigned during initialization of one instance.

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

One shared default; two independent readings



The readings do not become shared

Only the class-level formatting default is shared in this example.

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

NAZARBAYEV UNIVERSITY

Read the default through the class or instance

```
from shared_models import SensorFormat
SensorFormat.decimal_places = 2  # Establish this
     demonstration's initial state.
a = SensorFormat("A", 82.456)
b = SensorFormat("B", 315.678)
print(SensorFormat.decimal_places, a.
    decimal_places, b.decimal_places)
print(a.report())
print(b.report())
```

Observed output

```
2 2 2
A: 82.46
B: 315.68
```

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

Change a common reporting preference

```
from shared_models import SensorFormat
SensorFormat.decimal_places = 2
a = SensorFormat("A", 82.456)
b = SensorFormat("B", 315.678)
SensorFormat.decimal_places = 1  # Change the
    common default deliberately.
print(a.report())
print(b.report())
```

Observed output

```
A: 82.5
B: 315.7
```

What did not change?

Neither stored measurement was rounded or overwritten.

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

NAZARBAYEV UNIVERSITY

Instance assignment can shadow a class default

```
from shared_models import SensorFormat
SensorFormat.decimal_places = 2
a = SensorFormat("A", 82.456)
b = SensorFormat("B", 315.678)
a.decimal_places = 0  # Create an instance override,
    not a class change.
SensorFormat.decimal_places = 1
print(a.decimal_places, b.decimal_places, SensorFormat
    .decimal_places)
print(a.report())
print(b.report())
```

Observed output

```
0 1 1
A: 82
B: 315.7
```

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

NAZARBAYEV UNIVERSITY

Trace the lookup, not just the last assignment

Operation	Class	a reads	b reads
Start with default 2	2	2	2
`a.decimal_places = 0`	2	0	2
`SensorFormat.decimal_places = 1`	1	0	1
`del a.decimal_places`	1	1	1

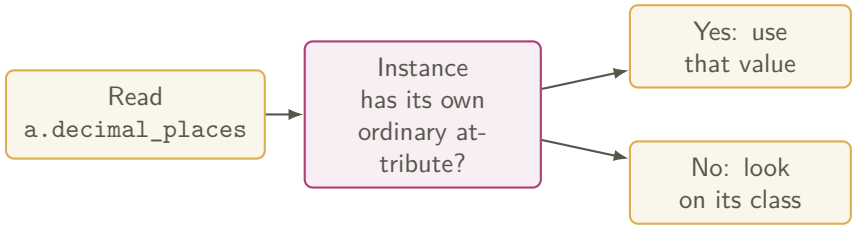
Why the last row changes a

Deleting the ordinary instance override exposes the class default again.

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

NAZARBAYEV UNIVERSITY

Ordinary attribute lookup: a useful local model



Scope of this diagram

Applies to the ordinary attributes shown here. Properties can intercept access; this is not the full Python attribute-resolution algorithm.

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

NAZARBAYEV UNIVERSITY

Choose the lookup policy inside a method

Always use the base class setting

`digits = SensorFormat.decimal_places`
All reports that use this exact expression follow that named class setting.

Allow local specialization

`digits = self.decimal_places`
This version permits an instance override and, later, a subclass default.

Teaching adaptation

The manuscript shows explicit class lookup; this deck also compares receiver-based lookup so shadowing is visible.

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

NAZARBAYEV UNIVERSITY

Prediction activity: find each attribute

```
# Predict ordinary attribute lookup after each
    assignment.
class Display:
    digits = 2
x = Display()
y = Display()
x.digits = 4
y.digits = 0
Display.digits = 1
print(x.digits, y.digits, Display.digits)
del x.digits  # Remove the override; future reads
     fall back to the class.
print(x.digits, y.digits)
```

Write before running

1. The first output line.
2. The second output line.
3. Which instance still has an override?

Individual then compare - 4 min

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

NAZARBAYEV UNIVERSITY

A method is not inside the class local scope

```
# Deliberate error: a method does not use the
    class namespace as a local scope.
class Display:
    digits = 2
    def precision(self):
        return digits  # Repair: self.digits or
    Display.digits.
print(Display().precision())
```

Expected failure

NameError: the unqualified name is not found.

Choose an explicit source

`self.digits` or `Display.digits`.

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

The shared-list trap

```
# Deliberate design error: these experiments were meant
      to be independent.
class Experiment:
    readings = []   # One class-level list is shared by
     both instances.
a = Experiment()
b = Experiment()
a.readings.append(10.0)   # Mutate the shared list; do
     not assign a new attribute.
b.readings.append(20.0)
print(a.readings)
print(b.readings)
print(a.readings is b.readings)
```

Predict the two histories

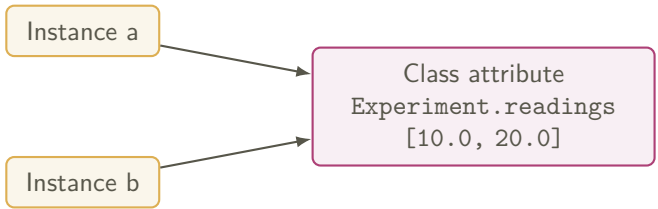
Would separate object creation make these histories independent?

Pairs - 3 min

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

NAZARBAYEV UNIVERSITY

Two instances can refer to one class-level list



The scientific risk

Readings from different experiments can be combined without any explicit data-processing step.

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

NAZARBAYEV UNIVERSITY

Create a new list during each initialization

```
class IndependentHistory:
    """Each instance owns a separate list; input values are finite numbers."""
    def __init__(self, name):
        self.name = name
        self.history = []  # A fresh list is created during every call.

    def add(self, value):
        self.history.append(checked_number(value, "Reading"))
```

Every constructor call evaluates [] again

Each instance receives its own mutable list. It may still be aliased deliberately, so tests remain useful.

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

NAZARBAYEV UNIVERSITY

Observe independent histories

```
from shared_models import IndependentHistory
a = IndependentHistory("A")
b = IndependentHistory("B")
a.add(10.0)
b.add(20.0)
print(a.history)
print(b.history)
print(a.history is b.history)  # Distinct lists:
    False.
```

Observed output

```
[10.0]
[20.0]
False
```

Identity check

`a.history is b.history` asks whether they are the same list, not whether their elements happen to match.

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

NAZARBAYEV UNIVERSITY

Mutation and rebinding are different operations

```
# Compare mutation of one shared object with assignment of a different object.
class Store:
    readings = []
a = Store()
b = Store()
a.readings.append(10.0)  # Both instances still look up the class list.
b.readings = [20.0]      # b now has its own instance attribute.
a.readings.append(30.0)
print(Store.readings)
print(a.readings)
print(b.readings)
```

Trace the class list and each instance - 3 min

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

A deliberate use of shared state: a counter

```
class CountedReading:
    """Count successful initializations of this standalone class in this run."""
    total_created = 0

    def __init__(self, value):
        # A rejected input must not increase the successful-creation count.
        new_value = checked_number(value, "Reading")
        self.value = new_value
        CountedReading.total_created += 1
        # This is not a count of currently live objects or update operations.
```

Meaning of this counter

Successful initializations of this standalone class during the current run. Validation occurs before incrementing it.

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

NAZARBAYEV UNIVERSITY

A failed creation should not count as success

```
from shared_models import CountedReading
CountedReading.total_created = 0  # Fresh,
    controlled test fixture.
a = CountedReading(10.0)
b = CountedReading(20.0)
try:
    CountedReading("bad")  # Validation fails
    before the counter update.
except TypeError:
    print("Invalid creation rejected")
print(CountedReading.total_created)
```

Observed output

```
Invalid creation rejected
2
```

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

NAZARBAYEV UNIVERSITY

Why self.total += 1 is not a class-wide count

```
# Deliberate counter bug: augmented assignment
    creates an instance attribute.
class Counter:
    total = 0
    def __init__(self):
        self.total += 1  # Reads class 0, then
    stores 1 on this instance.
a = Counter()
b = Counter()
print(a.total, b.total, Counter.total)  # Not a
    class-wide total of two.
```

Trace the augmented assignment

Read through self, add one, then assign the result back to that instance.

Expected output

```
1 1 0
```

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

NAZARBAYEV UNIVERSITY

Document what your counter does not measure

Quantity	Does `total_created` measure it?
Successful initializations in this run	Yes, by this standalone implementation
Currently live objects	No; deleting a name does not decrement it
Attempted creations including failures	No
Number of readings or updates	No
Unique IDs across computers or runs	No

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

NAZARBAYEV UNIVERSITY

Test independence and control shared settings

```
from shared_models import IndependentHistory
a = IndependentHistory("A")
b = IndependentHistory("B")
a.add(10.0)
assert a.history == [10.0]
assert b.history == []
assert a.history is not b.history
print("Independent history tests passed")
```

Repeatable shared-state tests

- Start from a known class default.
- Run a small isolated test.
- Restore changed settings afterward.
- Do not let test order choose the answer.

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

NAZARBAYEV UNIVERSITY

Design clinic: a laboratory with ten sensors

Choose where each item belongs

- A sensor name, unit, and current reading.
- A default number of report digits.
- Each sensor's measurement history.
- A documented total of successful creations.

Explain one decision

What would go wrong if each item were stored in the opposite place?

Pairs - 4 min

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

Lecture 26 exit ticket

Predict without running

A class has `digits = 2`. Instances `a` and `b` are created. Then `a.digits = 4` and `ClassName.digits = 1`. What does each instance read?

Explain

Where should a new per-experiment list be created? Why is append to a class list different from assigning a new instance list?

Individual - 3 min

Wang, Section 15.13 and Appendix H.9; Python Tutorial, Sections 9.3.5–9.4.

NAZARBAYEV UNIVERSITY

LECTURE 27

Inheritance and Specialized Engineering Classes

Reuse a clear interface; specialize behavior without breaking its promises

NAZARBAYEV UNIVERSITY

Lecture 27 learning goals

1. Recognize a genuine parent–child relationship.
2. Inherit behavior and initialize shared fields with super().
3. Add a specialized operation or override a method.
4. Trace which implementation a method call selects.
5. Use several sensor types through a common interface.
6. Test inherited contracts and distinguish is-a from has-a.

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

NAZARBAYEV UNIVERSITY

Warm-up: shared sensor responsibilities

Every sensor needs

- A name, unit, current value, and limits.
- A checked update.
- A query of the operating limits.
- A report that returns text.

Identify shared and specialized behavior - 3 min

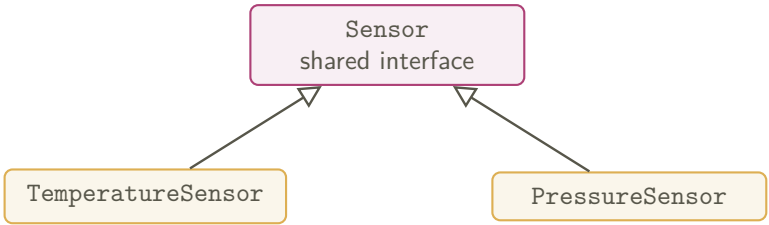
Some need additional operations

- Temperature: Celsius to Kelvin.
- Pressure: kPa to Pa.
- Voltage: V to mV.

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

NAZARBAYEV UNIVERSITY

Inheritance expresses a specialized kind of entity



Read the arrow toward the parent

A TemperatureSensor is a Sensor. The hollow arrowhead points to the more general class.

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

NAZARBAYEV UNIVERSITY

The smallest child inherits existing behavior

```
from sensor_hierarchy import Sensor
# No initializer here: the ordinary parent
    initializer is inherited.
class TemperatureSensor(Sensor):
    pass
s = TemperatureSensor("Tank", "deg C", 82.5, 0.0,
     90.0)
print(s.report())
print(isinstance(s, Sensor))
```

What is inherited?

- The initializer.
- The getter and update interface.
- The limit query and report.

A syntax-first version

It still requires the caller to supply the unit.

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

NAZARBAYEV UNIVERSITY

Start with the parent contract

Public interface	Meaning
`get_name()`, `get_unit()`	Return documented labels
`get_value()`, `get_limits()`	Return current value and inclusive limits
`set_value(x)`, `update(x)`	Accept a finite reading; preserve state on rejection
`is_safe()`, `report()`	Evaluate the rule; return Boolean or report text

Important scope

This hierarchy checks finite data and ordered limits, not full physical admissibility. It is not the absolute-pressure-only class.

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

NAZARBAYEV UNIVERSITY

The parent establishes shared fields

```
        # Store already-validated state on this same instance.
        self._name = name.strip()
        self._unit = unit.strip()
        self._value = reading
        self._lower_limit = lower
        self._upper_limit = upper
```

Before these assignments

The complete initializer checks nonempty string labels, finite numeric inputs, and ordered limits. See the included source file.

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

NAZARBAYEV UNIVERSITY

One update rule can serve the whole hierarchy

```
    def set_value(self, proposed_value):
        """Store a finite reading, even when outside operating limits."""
        new_value = checked_number(proposed_value, "Reading")
        self._value = new_value  # Commit only after validation succeeds.

    def update(self, proposed_value):
        """Keep the familiar Week 8 update interface; return None."""
        self.set_value(proposed_value)
```

A compatibility bridge

`update(x)` retains the familiar Week 8 method name while delegating to the explicit setter.

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

NAZARBAYEV UNIVERSITY

A query uses the receiver, including a child

```
    def is_safe(self):
        """Return whether the current reading
    meets both inclusive limits."""
        return self._lower_limit <= self.
    get_value() <= self._upper_limit

    def status(self):
        """Return an interpretation of the stated
     rule, not physical safety."""
        if self.is_safe():
            return "WITHIN LIMITS"
        return "OUTSIDE LIMITS"
```

Receiver is still the child

An inherited method receives the actual child instance as self. There is no separate hidden parent sensor.

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

NAZARBAYEV UNIVERSITY

Specialize initialization with super()

```
class TemperatureSensor(Sensor):
    """Specialize the unit and add Celsius-to-Kelvin conversion."""
    def __init__(self, name, value, lower_limit, upper_limit):
        # Initialize inherited fields on this SAME new child instance.
        super().__init__(name, "deg C", value, lower_limit, upper_limit)

    def to_kelvin(self):
        """Return the converted value; stored Celsius reading is unchanged."""
        return self.get_value() + 273.15
```

What is new?

The caller no longer supplies the unit, and the child adds a temperature conversion.

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

Trace child construction step by step

Step	What happens
`TemperatureSensor("Tank", 82.5, 0, 90)`	A new child instance is created
Child initializer receives self	The receiver is that new TemperatureSensor
`super().__init__(...)`	Parent validates and initializes the same receiver
Child initializer finishes	Caller receives the fully initialized child

Not another object

`super()` does not construct a separate Sensor. It provides access to inherited behavior.

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

NAZARBAYEV UNIVERSITY

Test conversions and unchanged state

```
from sensor_hierarchy import TemperatureSensor, PressureSensor
t = TemperatureSensor("Ice point", 0.0, -10.0, 90.0)
p = PressureSensor("Test pressure", 2.0, 0.0, 4.0)
assert abs(t.to_kelvin() - 273.15) <= 1e-12
assert p.to_pa() == 2000.0
assert t.get_value() == 0.0  # Conversion must not overwrite Celsius storage.
assert p.get_value() == 2.0  # Conversion must not overwrite kPa storage.
print("Conversion and state tests passed")
```

Two types of evidence

The converted quantity is right, and the original stored representation is unchanged.

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

NAZARBAYEV UNIVERSITY

A pressure child adds its own operation

```
    def to_pa(self):
        """Return the value in Pa; extreme overflow is rejected explicitly."""
        result = self.get_value() * 1000.0
        return checked_number(result, "Converted pressure in Pa")
```

The initializer supplies kPa

```
super().__init__(name, "kPa", value, low, high)
```

Preserve meaning

Multiplying by 1000 changes the numerical unit, not whether the pressure is absolute or gauge.

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

Override a method without breaking its promise

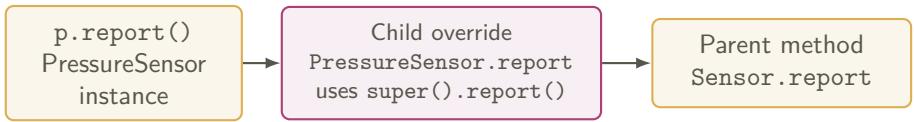
```
    def report(self):
        """Extend, rather than discard, the inherited report contract."""
        base_text = super().report()  # This does not call this override again.
        try:
            converted_text = f"{self.to_pa():.0f} Pa"
        except ValueError:
            # Keep reporting the valid kPa reading if its conversion overflows.
            converted_text = "Pa conversion unavailable"
        return f"{base_text} | {converted_text}"
```

Same name, compatible behavior

The child returns text without changing the reading. If conversion overflows, it keeps the kPa report and explicitly marks the unavailable Pa value.

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

Which report method runs?



Return path

The parent returns its text. The child adds its extra text, then returns the complete report to the caller.

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

NAZARBAYEV UNIVERSITY

Polymorphism: one loop, different implementations

```
from sensor_hierarchy import Sensor, TemperatureSensor, PressureSensor
sensors = [
    TemperatureSensor("Tank", 82.5, 0.0, 90.0),
    PressureSensor("Pipe", 315.0, 100.0, 400.0),
    Sensor("Battery", "V", 10.8, 11.0, 12.8),
]
# The loop uses a shared interface, not repeated exact-type branches.
for sensor in sensors:
    print(sensor.report())
```

Common interface

The loop asks every object for a report. Each object supplies the applicable implementation.

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

NAZARBAYEV UNIVERSITY

Inspect exact type and inheritance separately

```
from sensor_hierarchy import Sensor, TemperatureSensor
    , PressureSensor
t = TemperatureSensor("Tank", 20.0, 0.0, 90.0)
print(type(t) is TemperatureSensor)  # Exact runtime
    type.
print(type(t) is Sensor)             # Exact type is
    not the parent.
print(isinstance(t, Sensor))         # A child instance
     is also a Sensor.
print(issubclass(TemperatureSensor, Sensor))
print(isinstance(t, PressureSensor)) # A sibling
    relationship is different.
```

Observed output

```
True
False
True
True
False
```

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

NAZARBAYEV UNIVERSITY

Class defaults can also be specialized by a child

```
from sensor_hierarchy import Sensor, TemperatureSensor
    , PressureSensor
Sensor.decimal_places = 1
PressureSensor.decimal_places = 0
t = TemperatureSensor("Tank", 20.0, 0.0, 90.0)
p = PressureSensor("Pipe", 315.0, 100.0, 400.0)
Sensor.decimal_places = 2  # Temperature inherits this;
     pressure shadows it.
print(t.decimal_places, p.decimal_places, Sensor.
    decimal_places)
p.decimal_places = 3      # Now one pressure instance
    overrides its child class.
print(p.decimal_places, PressureSensor.decimal_places)
```

Observed output

```
2 0 2
3 0
```

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

NAZARBAYEV UNIVERSITY

A child must remain usable through the parent contract

Preserve the promised behavior

- Compatible inputs and returned types.
- The same meaning of units and operations.
- No unexpected mutation in a query.
- Documented failure behavior.

Inspect example 26 and explain its returned value

A misleading override

If `report()` prints instead of returning text, client code receives None. Reusing the name alone is not enough.

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

NAZARBAYEV UNIVERSITY

Test inherited behavior as well as new methods

```
from sensor_hierarchy import PressureSensor
p = PressureSensor("Pipe", 315.0, 100.0, 400.0)
p.update(400.0)  # The inherited inclusive boundary is accepted.
assert p.is_safe()
p.update(401.0)  # Valid reading outside the operating rule: retain and flag it.
assert not p.is_safe()
before = p.get_value()
try:
    p.update("bad")
except TypeError:
    assert p.get_value() == before
else:
    raise AssertionError("Expected a TypeError")
print("Inherited behavior tests passed")
```

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

NAZARBAYEV UNIVERSITY

Debugging clinic: the parent was never initialized

```
from sensor_hierarchy import Sensor
# Deliberate error: required parent attributes are
    never initialized.
class TemperatureSensor(Sensor):
    def __init__(self, name, value):
        self._unit = "deg C"  # Not a replacement for
    the parent's initialization.
t = TemperatureSensor("Tank", 20.0)
print(t.get_value())  # _value does not exist.
```

Identify the first missing fact

1. Which initializer actually ran?
2. Which required attribute was never stored?
3. Where should `super().__init__` be called?

Pairs - 4 min

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

Is-a is different from has-a

Inheritance

A `PressureSensor` is a `Sensor`. It should support the sensor interface.

Composition

An `Experiment` has a list of sensors. It does not become one of those sensors.

Relationship	A useful model
Temperature sensor / sensor	Specialization: inheritance
Bar / material	Containment: composition
Experiment / sensors	Containment: composition

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

Design activity: add a VoltageSensor

Reuse before rewriting

1. Inherit from `Sensor`.
2. Supply the unit `"V"` through `super().__init__`.
3. Add `to_millivolts()` without changing stored volts.
4. Test 12 V $\rightarrow$ 12,000 mV and an inherited limit.

Pairs - 5 min

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

NAZARBAYEV UNIVERSITY

Integrate and retain engineering evidence

A review of the sensor family

- Each label and unit has a meaning.
- The parent defines a shared contract.
- Children add conversions and reporting.
- Each instance retains separate readings.

Evidence to report

- At least one independently known answer.
- A boundary and a failed update.
- An independence check.
- A statement of modeling limitations.

Practice only; no new graded task or Moodle policy is introduced.

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

Lecture 27 exit ticket

Read and explain

1. What does `super().__init__(...)` initialize?
2. Why can `isinstance(t, Sensor)` be true when its exact type is TemperatureSensor?
3. Why must a reporting override still return a string?
4. An experiment owns sensors: is-a or has-a?

Individual - 3 min

Wang, Sections 15.18–15.24; Appendix H.22–H.27; Guttag, Section 8.2.

NAZARBAYEV UNIVERSITY

Optional: attribute-like access with a property

```
class PressureProperty:
    """One absolute pressure in kPa, exposed through a validated property."""
    def __init__(self, value_kpa):
        self.value_kpa = value_kpa  # Assignment invokes the property setter.

    @property
    def value_kpa(self):
        """Return the current pressure in kPa."""
        return self._value_kpa

    @value_kpa.setter
    def value_kpa(self, proposed_kpa):
        # Use a DIFFERENT backing name; self.value_kpa would recurse here.
        new_value = checked_nonnegative(proposed_kpa, "Absolute pressure")
        self._value_kpa = new_value
```

Wang, Appendix H.13; Python built-in property() documentation. Optional extension.

Optional: properties need separate backing data

```
from optional_models import PressureProperty
p = PressureProperty(315.0)
p.value_kpa = 400.0  # Attribute-like syntax invokes
    the setter.
print(p.value_kpa)   # Attribute-like syntax invokes
    the getter.
try:
    p.value_kpa = -1.0
except ValueError:
    print("Rejected")
print(p.value_kpa)   # Validated property preserved the
     previous reading.
```

Common recursion mistake

Inside the setter, `self.value_kpa = proposed_kpa` would invoke that setter again. Store to `self._value_kpa` after validation.

A property controls the public route, not access to internal backing names.

Wang, Appendix H.13–H.14; Python property() documentation. Optional extension.

NAZARBAYEV UNIVERSITY

Reference: the complete finite-number helper

```python
import math

def checked_number(value, label):
    """Return a finite float from a built-in int or float, excluding bool."""
    # A Boolean is an int subclass but not a measurement in this contract.
    if isinstance(value, bool) or not isinstance(value, (int, float)):
        raise TypeError(f"{label} must be an int or float, not bool")
    try:
        result = float(value)  # Normalize the accepted numeric representation.
    except OverflowError as error:
        raise ValueError(f"{label} is too large for a float") from error
    if not math.isfinite(result):  # Reject NaN and positive/negative infinity.
        raise ValueError(f"{label} must be finite")
    return result
```

Week 7 validation tools; Wang, Appendix H.12. Explicit teaching-domain checks.

Optional: see what name mangling actually does

```
# Name mangling reduces accidental name
    collisions; it is not security.
class Device:
    def __init__(self):
        self.__code = "A"
    def get_code(self):
        return self.__code
item = Device()
print(item.get_code())
print(item._Device__code)  # Demonstrates
    accessibility, not recommended client use.
```

Observed output

```
A
A
```

Purpose, not secrecy

A double-leading underscore changes the internal name to reduce accidental collisions. It does not prevent a determined caller from accessing it.

Wang, Appendix H.14; Python Tutorial, Section 9.6. Optional clarification.

Optional: readable and diagnostic text

```
    def __str__(self):
        """Supply readable text for print(
    instance)."""
        return f"Absolute pressure: {self.
    value_kpa:.1f} kPa"

    def __repr__(self):
        """Supply a diagnostic description for
    repr(instance)."""
        return f"PressureProperty({self.value_kpa
    !r})"
```

Calling examples

```
Absolute pressure: 315.0 kPa
PressureProperty(315.0)
```

Prefer predictable behavior

`__str__` and `__repr__` return text; they should not unexpectedly change the reading.

Wang, Section 15.14 and Appendix H.17. Optional extension.

NAZARBAYEV UNIVERSITY

Reference: interface and inheritance quick guide

Syntax	Interpretation
`obj.get_value()`	Call a public getter
`obj.set_value(x)`	Request a controlled update
`ClassName.setting`	Read or change a class attribute
`class Child(Parent):`	Declare a specialized type
`super().method(...)`	Continue lookup to inherited behavior
`isinstance(obj, Parent)`	Accept a parent or subclass instance
`self._value`	Access a non-public backing attribute

Course Specification Session 9; Wang, Ch. 15 and Appendix H; Python documentation.

NAZARBAYEV UNIVERSITY

Suggested reading and source basis

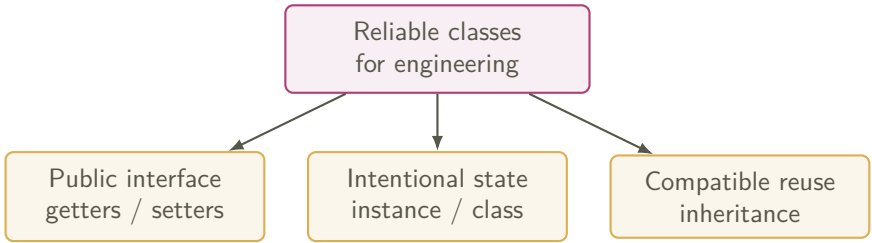
- **Course scope:** NU, ENG 101 Course Specification Report, Session 9; CLOs 1 and 5.
- **Engineering examples:** Y. Wang, *Programming for Engineers with Python*, supplied manuscript v20260815, Sections 15.13, 15.18–15.24; Appendix H.8–H.14 and H.22–H.27.
- **Abstraction and inheritance:** J. V. Guttag, 2nd ed., Sections 8.2–8.3.
- **Technical checks:** Python Tutorial, Sections 9.3.5–9.6; built-in `property()` and `super()` documentation.

Adaptation note

New examples and diagrams are teaching adaptations. Optional features are labelled. Operating limits and data are illustrative, not design specifications.

Supplied course sources; official Python documentation checked 14 September 2026.

Week 9 synthesis and the next curriculum unit



Next: algorithmic efficiency

The next specified unit studies orders of growth and Big-O notation. Correctness and clear contracts remain prerequisites.

ENG 101 Course Specification, Sessions 9–10; synthesis of this week.

NAZARBAYEV UNIVERSITY

A reusable class keeps its promises.

Make state changes intentional.
Make shared data visible.
Make specializations compatible.

ENG 101 Week 9 Yanwei Wang